

POLYSOURCE

Rapport d'avancement du projet

ELSE4 FISA — Polytech Nice Sophia — Année 2025-2026

Système de surveillance d'un bassin de montagne

Mesure du débit de débordement et du niveau d'eau

Équipe projet : Andy CLÉMENT | Baptiste LECLERC | Ibadete FETIU | Matthis GOMEZ

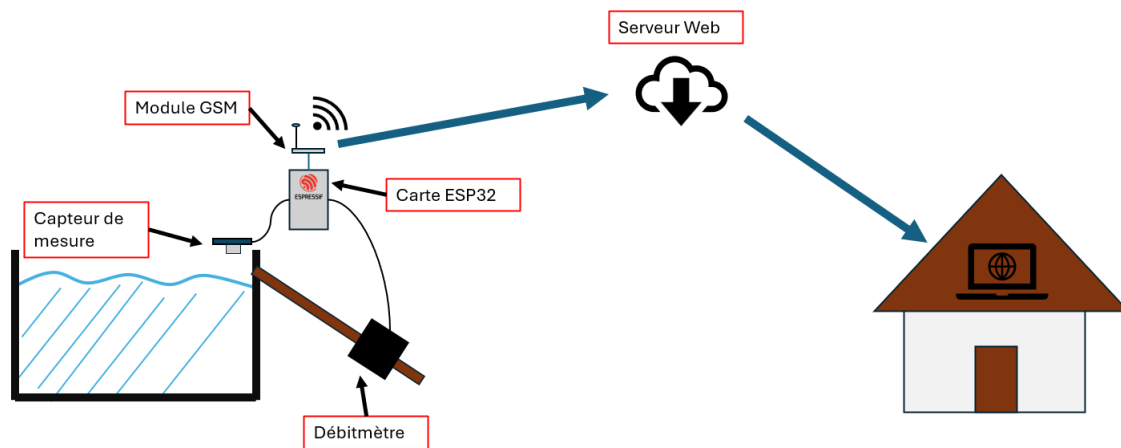
Mai 2026

1. Description du projet

PolySource est un système IoT de surveillance destiné à mesurer en temps réel le débit de débordement et le niveau d'eau d'un bassin de montagne. L'objectif est de permettre à un utilisateur distant de consulter ces données via une interface web, même depuis une zone isolée sans réseau filaire.

Le dispositif est conçu pour être autonome énergétiquement (alimenté par batterie), et communique via le réseau mobile 4G. Le schéma d'ensemble repose sur la chaîne suivante : capteurs physiques → ESP32 → module GSM 4G → broker MQTT HiveMQ Cloud → Raspberry Pi 5 (stack TIG) → interface Grafana.

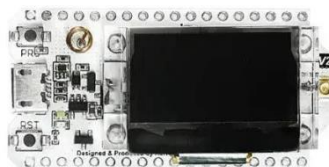
Voici pour illustrer un dessin récapitulatif :



a) Matériel utilisé

Voici ci-joint le matériel utilisé :

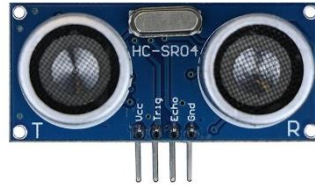
- ESP32 Heltec WiFi LoRa 32 :
Son rôle est de traiter les données et les mettre sous format JSON. C'est une puce puissante avec une consommation réduite (5V) et une programmation simple grâce à l'Arduino IDE



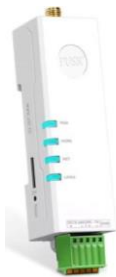
- Débitmètre DIGITEN G3/4 :
Il permet la mesure du débit de débordement du bassin et ainsi estimer le trop plein du bassin. Il peut prendre des mesures sur l'intervalle 1–60 L/min. Son avantage est qu'il peut se raccorder aux broches de l'ESP32 et qu'il a une consommation réduite de courant (3-24V).



- Capteur Ultrasons HC-SR04 :
Il permet de mesurer le niveau d'eau contenue dans le bassin. Il consomme peu de courant et se branche facilement avec l'ESP32



- Module GSM USR-DR154 / Module GSM WH-LTE-7S1-E :
Ces modules GSM se connectent aux réseaux 4G à l'aide d'une SIM. Leur rôle est de communiquer les données du bassin aux serveurs distants avec une transmission MQTT. Ces modules fonctionnent avec 12V, ce qui nécessite une alimentation externe.



USR-DR145



WH-LTE-7S1-E

- Raspberry Pi 5 :
Carte électronique qui héberge les données reçues et qui est chez l'utilisateur. Il gère toute l'interface graphique du projet



- Adaptateur USB-RS485 :
Permet de relier l'USR-DR154 au port USB du PC
- Adaptateur RS232-RS485 :
Permet la communication entre l'ESP32 et le GSM USR-DR154.

b) Découpage des tâches

Le groupe a été divisé en deux pôles de travail distincts : la partie hardware/firmware qui est occupé par Andy et Matthis et la partie software/visualisation occupé par Baptiste et Ibadete. Voici un tableau récapitulatif des différentes tâches de chacun :

Membre(s)	Responsabilités
Andy CLÉMENT & Matthis GOMEZ	Partie hardware : lecture des capteurs (débitmètre, capteur ultrasons), intégration ESP32, formatage JSON, configuration et communication GSM/MQTT
Baptiste LECLERC	Partie serveur : déploiement de la stack TIG sur Raspberry Pi 5 (Docker, Telegraf, InfluxDB, Grafana), automatisation et scripting
Ibadete FETIU	Partie visualisation : développement et optimisation des dashboards Grafana (panels ECharts, indicateur de débordement, responsive design)

2. Affichage sur site Web

La solution de visualisation choisie est Grafana, déployé sur un Raspberry Pi 5 via Docker. La stack technique retenue est la stack T.I.G. (Telegraf, InfluxDB, Grafana), qui constitue une architecture réputée pour les projets IoT.

Stack technique déployée

- Telegraf : collecteur de données qui s'abonne au topic MQTT (source/mesures) via HiveMQ Cloud et insère les données dans InfluxDB.
- InfluxDB : base de données de séries temporelles qui stocke les mesures horodatées (niveau d'eau, débit, état de la batterie, overflow).
- Grafana : outil de visualisation qui lit les données depuis InfluxDB et les affiche sous forme de graphiques dynamiques.

Dashboard Grafana

Le Dashboard Grafana a été développé par Baptiste et Ibadete. Il comprend les panneaux suivants :

- Graphique d'évolution du niveau d'eau au cours du temps
- Graphique d'évolution du débit de débordement
- Indicateur du niveau d'eau courant (visualisation dynamique)
- Indicateur du niveau de batterie
- Panneau de détection de débordement (état normal / overflow), développé avec le plugin ECharts

L'interface a été rendue responsive pour s'adapter à différentes tailles d'écran (usage terrain sur mobile).

Déploiement automatisé

Baptiste a développé un script `setup.sh` qui automatise l'installation complète de la stack sur un Raspberry Pi vierge (installation Docker, configuration des répertoires, permissions Grafana, lancement des conteneurs). La configuration est déclarée dans un fichier `docker-compose.yml` et les secrets (tokens InfluxDB, identifiants HiveMQ) sont centralisés dans un fichier `.env` ignoré par Git.

Le provisioning automatique de Grafana permet de retrouver l'ensemble des sources de données et des dashboards sans aucune intervention manuelle dans l'interface web, garantissant ainsi la reproductibilité du déploiement.

Validation End-to-End

Lors de la séance du 10/04/2026, la chaîne complète a été validée : réception par le broker HiveMQ Cloud → parsing Telegraf → archivage InfluxDB → affichage en temps réel sur Grafana. Cette validation confirme le bon fonctionnement de l'ensemble de la stack de visualisation.

Configuration de la Raspberry Pi (Reprise à zéro)

Pour reconfigurer la Raspberry Pi de zéro, voici la marche à suivre :

Télécharger et installer Raspberry Pi imager :

Setup steps

Device

OS

Stockage

Customisation

Writing

Done

APP OPTIONS

Select your Raspberry Pi device



Raspberry Pi 5
Raspberry Pi 5, 500 / 500+, and Compute Module 5



Raspberry Pi 4
Raspberry Pi 4 Model B, 400, and Compute Module 4 / 4S



Raspberry Pi 3
Raspberry Pi 3 Model A+ / B / B+ and Compute Module 3 / 3+



Raspberry Pi 2
Raspberry Pi 2 Model B



Raspberry Pi 1
Raspberry Pi 1 Model A / A+ / B / B+ and Compute Module 1

SUIVANT

<https://www.raspberrypi.com/software/>

Sélectionner le modèle de carte (ici Raspberry Pi 5)

Pour l'OS j'ai choisi l'os recommandé "Raspberry Pi OS (64-bits)", il pourrait être intéressant de comparer avec d'autres OS comme un OS server, mais celui-ci à l'inconvénient de ne pas avoir d'interface graphique. Pour une première approche l'interface graphique ce révèle être d'une grande aide.

Ensuite il faut sélectionner la carte micro SD à utiliser. Pour la partie customisation, utiliser les différentes informations du fichier `logins.md`.

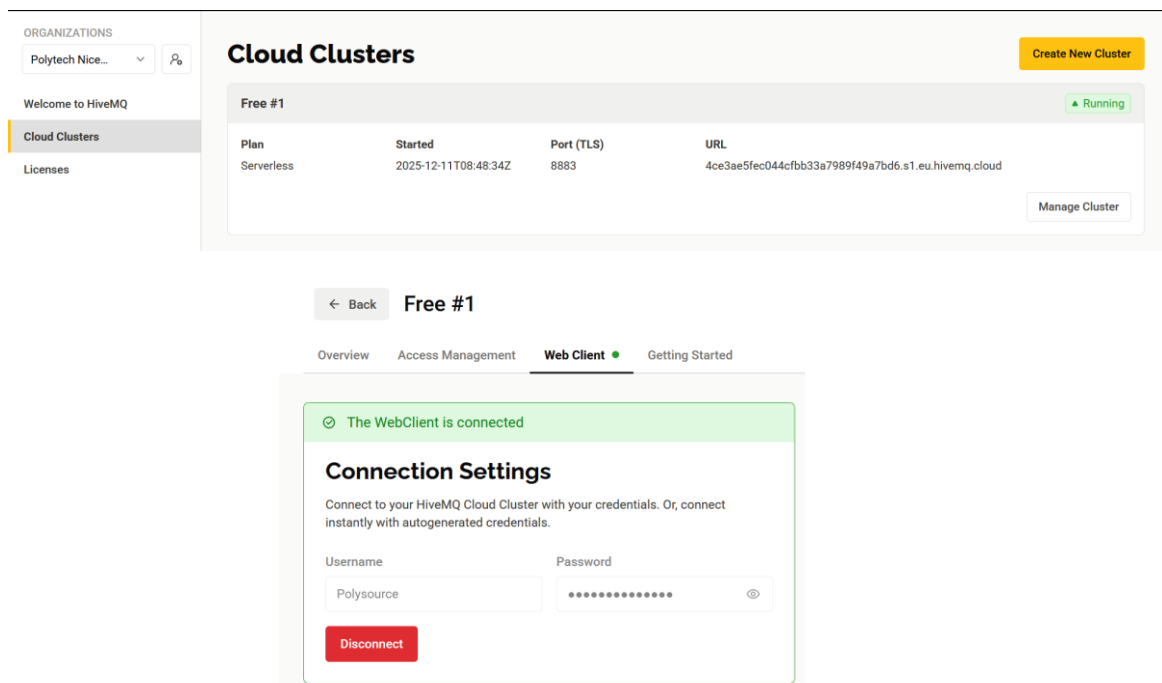
Il ne reste plus qu'à finir l'installation. Une fois l'installation terminée, remettre la carte micro

SD dans la Raspberry pi puis suivre les étapes décrites dans RESSOURCES/INIT_RASPi.md.

Validation de la chaîne de données via HiveMQ Cloud

Afin de valider le bon fonctionnement de l'infrastructure sans dépendre du matériel de l'ESP32, il est possible de simuler l'envoi de trames IoT :

1. Se connecter à la console : <https://console.hivemq.cloud/> (Identifiants dans *logins.md*).
2. Cliquer sur **Manage Cluster**, puis aller sur l'onglet **Web Client** et s'authentifier à nouveau avec les accès du broker.



3. Via l'onglet **Topic Subscription**, s'abonner au topic cible : source/mesures.
4. Via l'onglet **Publish (Send Message)**, envoyer une charge utile simulant l'ESP32.

Exemple de message à envoyer (pensez à actualiser le timestamp Unix):

```
{
  "device_id": "source01",
  "location" : "spring_A",
  "timestamp": 1708185600,
  "water_level_cm" : 45.2,
  "flow_rate_l_min" : 12.5,
  "battery_percent" : 88,
  "overflow" : 0
}
```

Diagnostic et Débogage

Si les données publiées n'apparaissent pas sur les graphiques Grafana, la chaîne est rompue. Il faut alors inspecter les logs des conteneurs pour localiser le problème (erreur de parsing Telegraf, mauvaise route d'UID dans Grafana, etc.) à l'aide des commandes suivantes dans le terminal de la Raspberry Pi :

```
cd /opt/tig_stack  
sudo docker compose logs telegraf  
sudo docker compose logs grafana
```

3. Relevé des données capteurs et code

Dans cette partie, détaillons les différentes démarches réalisées pour relever les données des capteurs et les exploiter à l'aide de l'ESP-32

Mesure du débit : Débitmètre DIGITEN G3/4

Le débitmètre est branché sur le port GPIO 23 de l'ESP32. Le capteur est branché sur ce port car le débitmètre génère des impulsions PWM proportionnelles au débit mesuré grâce à la rotation d'une hélice qui se situe à l'intérieur du débitmètre. La relation entre fréquence d'impulsions (F) et débit volumique (Q) est donnée par la formule constructeur :

$$F = 5.5 \times Q \text{ (en L/min)}$$

En en duit que la valeur du débit se note :

$$Q = \frac{F}{5.5} \text{ (en L/min)}$$

Le code utilise les interruptions hardware de l'ESP32 : à chaque front montant détecté sur le pin 23, un compteur est incrémenté. Toutes les secondes, le débit est calculé puis le compteur remis à zéro. Cette approche garantit une mesure précise sans bloquer l'exécution principale du programme. On stock alors la valeur relevée dans une variable qui permettra de l'utiliser dans le JSON

Relevé du Niveau : Sonar HC-SR04

Le capteur ultrasonore est configuré sur les GPIO 35 (Trigger) et 34 (Echo). Il mesure la distance jusqu'à la surface de l'eau en chronométrant le temps d'aller-retour d'une impulsion ultrasonore. La distance est calculée selon :

$$D = \frac{T_e}{2} \times 0,034$$

Avec D la distance et T_e la durée de l'écho généré par le code. Un timeout de 500 ms est appliqué pour éviter les blocages en cas d'absence de retour écho. Un correctif hardware a été apporté (impulsion LOW de 5 μ s avant chaque mesure) pour stabiliser les lectures. Un seuil d'alerte avec hystérésis (+/- 2 cm) permet de détecter et signaler un niveau critique sans oscillations intempestives.

Note : des problèmes de lectures instables ont été rencontrés lors de l'intégration complète du code (suspicion de faux contact hardware). Ce point a été résolu lors de la dernière séance par Matthis.

Formatage JSON et transmission

Pour la lecture des données et l'envoi vers le Broucker MQTT, il a été décidé que les données devaient être envoyées sous format JSON. Les données des deux capteurs sont alors agrégées par l'ESP32 et sérialisées en JSON via la bibliothèque ArduinoJson by Benoit Blanchon. La trame transmise au module GSM contient les champs suivants :

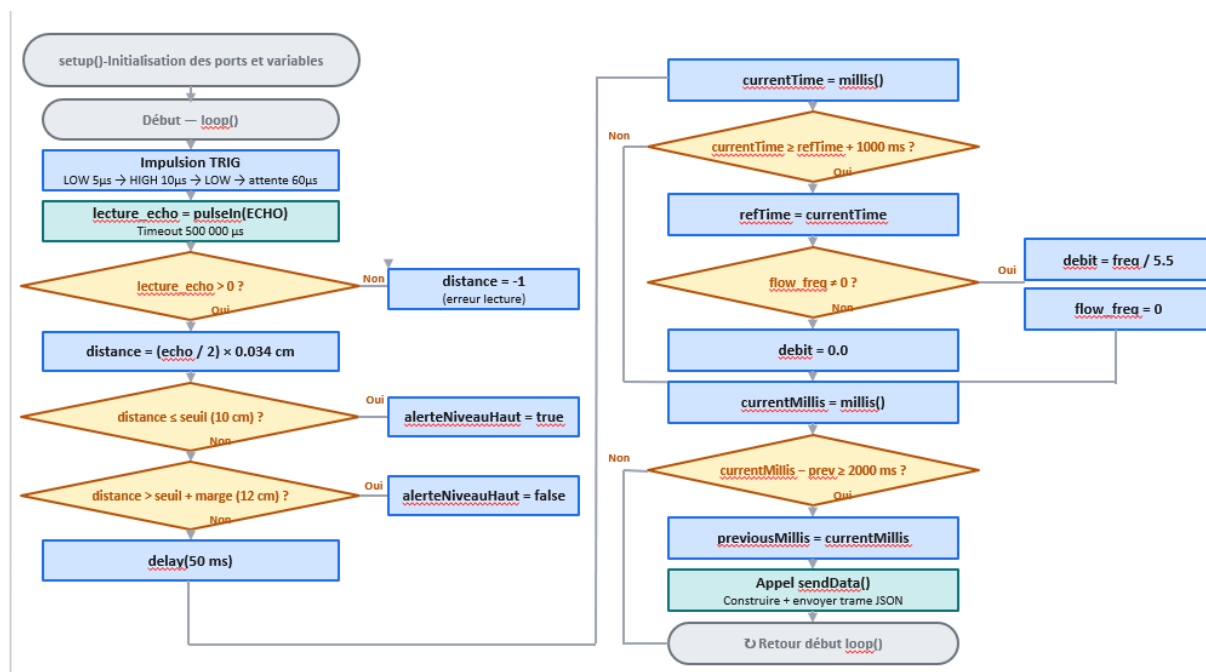
- device_id : identifiant du dispositif
- location : localisation de la source
- timestamp : horodatage Unix (voir ci-dessous)
- water_level_cm : niveau d'eau mesuré par le sonar
- flow_rate_l_min : débit mesuré par le débitmètre

- battery_percent : niveau de batterie (géré par un projet externe)
- overflow : booléen indiquant un état de débordement du bassin

Bien que toutes ces données sont assez simples à calculer, la valeur « timestamp » reste un point technique à soulever. En effet, L'ESP32 ne dispose pas de RTC (horloge temps réel) intégrée. L'une des solutions envisagées consiste à récupérer l'heure réseau via la commande `AT+CCLK?` du module USR-DR154, synchronisée avec le réseau GSM, puis à utiliser la fonction `millis()` pour calculer le temps écoulé depuis le démarrage. La combinaison des deux donne un timestamp Unix suffisamment précis pour l'archivage dans InfluxDB. Une autre solution envisagée est d'utiliser seulement la fonction `millis()` de l'ESP32 et d'utiliser la valeur du temps écoulé depuis le démarrage de l'ESP 32 en tant que « timestamp ». Bien que plus simple à exécuter, un problème se pose lorsque l'ESP32 redémarre. Les données enregistrées précédemment seraient supprimées. Ces solutions sont à étudier.

Code de l'ESP32

Avec toutes les contraintes vues dans les parties précédentes, voici un algorithme du code actuel :



4. Configuration GSM et connexion au broker MQTT

Matériel de communication

Deux modules GSM ont été évalués au cours du projet : le WH-LTE-7S1-E et l'USR-DR154, tous deux fabriqués par PUSR. Une grande partie du projet a été passé dans la configuration du WH-LTE-7S1-E mais a été écarté du projet. Ce module n'avait pas de commande `AT+MQTT` dans sa bibliothèque, ce qui rendait difficile sa configuration. L'USR-DR154 a été retenu comme module pour ce projet car plus simple à déboguer et à configurer. Contrairement au WH-LTE-7S1-E, l'USR-DR154 communique grâce au protocole RS485.

L'ESP32 ne supportant pas le protocole RS485 nativement, deux adaptateurs de conversion ont été nécessaires :

- USB → RS485 : pour la configuration du module depuis un PC via le logiciel PV-DTUSet (disponible sur GitHub)
- RS232 → RS485 : pour la communication en production entre l'ESP32 et le module GSM

Connexion au réseau 4G

Le module GSM est configuré avec une carte SIM Free fournie par l'encadrant du projet. La connexion au réseau 4G (LTE) est vérifiée via les commandes AT suivantes :

- AT+CSQ (qualité signal) : si le signal est compris entre 10 et 15, on peut considérer que le signal est correct. Plus la valeur est grande, plus fort est le signal.
- AT+SYSINFO (type réseau) : Permet de savoir si le module est connecté au réseau 4G, LTE. Renvoie 2 si c'est le cas.
- AT+CIP (adresse IP attribuée) : Affiche l'adresse IP attribuée au GSM

Pour que le GSM puisse se connecter au réseau 4G, la SIM doit bien être reconnu par le GSM, pour cela l'APN doit être configuré en "free" et la SIM ne doit pas être protégé par un mot de passe.

Configuration MQTT et déblocage firmware

La connexion au broker MQTT HiveMQ Cloud (port 8883, TLS) a représenté le défi technique majeur du projet. Pendant plusieurs séances, le module WH-LTE-7S1-E refusait de se connecter pour une raison inconnue. Même en essayant de le configurer manuellement avec les commandes AT, impossible d'établir la connexion. Dans un deuxième temps, nous avons essayé le module USR-DR154 cependant sans la commande AT+SSLCFG — indispensable pour configurer la couche TLS — la connexion ne pouvait pas s'établir. La commande n'était pas reconnue par le firmware alors qu'elle était inscrite dans la bibliothèque du constructeur.

Par suite d'un contact avec le support technique PUSR, une mise à jour du firmware a été obtenue et appliquée avec succès lors de la séance du 10/04/2026. Une séquence de configuration MQTT complète a été établie :

```
AT+WKMOD=MQTT,NOR //On met le GSM en mode MQTT
AT+MQTTSVR=<adresse>.eu.hivemq.cloud,8883 //On configure l'adresse du serveur
AT+MQTTUSER=<identifiant> //On entre l'identifiant du Cluster
AT+MQTTPSW=<mot_de_passe> //On entre le mot de passe du Cluster
AT+MQTTCID=USR-DR154 //Identifiant utilisé par le GSM
AT+SSLEN=ON
AT+SSLCFG=ON
AT+SSLAUTH=NONE
```

Après cette configuration, le module s'est connecté au broker HiveMQ Cloud et des messages ont pu être publiés sur le topic *source/mesures* et reçus avec succès. La chaîne de communication complète (capteurs → ESP32 → GSM → MQTT → Grafana) a ensuite été validée en conditions réelles.

Un tutoriel détaillé de configuration du module USR-DR154 a été rédigé et intégré au dépôt GitHub (Ressource/Tuto/Tuto-USR-DR154.md) pour faciliter la reprise du projet par de futurs groupes.

5. Ce qu'il reste à faire

Bien que le projet ait atteint un niveau de fonctionnement satisfaisant à la fin de l'année, plusieurs axes d'amélioration et tâches non réalisées ont été identifiés :

Partie hardware et firmware

- Optimisation de la gestion de l'énergie : le projet devant fonctionner sur batterie en autonomie, une gestion fine de la consommation (deep sleep ESP32, intervalles d'envoi adaptatifs) reste à implémenter.
- Intégration sur PCB dédiée : le prototype actuel repose sur des cartes et câblages séparés. La conception d'un PCB intégrant l'ESP32 et les connecteurs serait une étape de maturité importante.

Partie visualisation et serveur

- Système d'alertes automatiques : une réflexion a été engagée sur des alertes en cas de dépassement de seuil (niveau d'eau critique, batterie faible), mais cette fonctionnalité n'a pas été finalisée dans Grafana.
- Stabilité du support de stockage : la carte SD du Raspberry Pi a subi deux défaillances critiques au cours du projet. L'utilisation d'un SSD externe en remplacement de la carte SD serait fortement recommandée.
- Tests d'affichage multi-écrans : le responsive design du dashboard a été initié mais nécessite des tests complémentaires sur différentes résolutions et terminaux.
- Optimisation des performances Grafana : des ralentissements ont été observés lors de l'utilisation intensive de l'interface graphique. Des optimisations (réduction du taux de rafraîchissement, simplification des panels complexes) sont à envisager.

Tests en conditions réelles

- Déploiement sur le terrain : le système a été validé en laboratoire mais n'a pas encore été déployé sur un bassin réel en montagne.
- Test d'autonomie énergétique : la durée de vie sur batterie dans des conditions réelles (variations thermiques, connexions intermittentes) reste à évaluer.

6. Conclusion

Le projet PolySource a permis de concevoir et de valider un système IoT complet de surveillance d'un bassin de montagne, couvrant l'ensemble de la chaîne technique : acquisition des données physiques, traitement embarqué, transmission sans fil via 4G/MQTT, stockage en base de données et visualisation en temps réel.

Les principaux résultats obtenus à l'issue du projet sont les suivants :

- Acquisition fonctionnelle des données des deux capteurs (débitmètre DIGITEN et sonar HC-SR04) par l'ESP32, avec formatage JSON et horodatage.
- Connexion réussie du module GSM USR-DR154 au broker MQTT HiveMQ Cloud (après mise à jour firmware obtenue auprès du support PUSR), permettant la publication des trames JSON sur le topic source/mesures.
- Déploiement d'une stack TIG (Telegraf, InfluxDB, Grafana) sur Raspberry Pi 5, entièrement conteneurisée via Docker et automatisée par script, permettant un redéploiement en quelques minutes.
- Interface Grafana complète et fonctionnelle, avec affichage en temps réel du niveau d'eau, du débit, de l'état de la batterie et d'un indicateur de débordement.
- Validation end-to-end de la chaîne complète en conditions quasi-réelles.

Le principal défi du projet a été la configuration du module GSM pour la connexion MQTT sécurisée (TLS), qui a bloqué l'équipe pendant plusieurs séances avant d'être résolu grâce à une mise à jour firmware. Cet épisode illustre la complexité inhérente aux projets IoT embarqués, où les contraintes matérielles peuvent générer des blocages imprévus.

Pour les groupes futurs, un tutoriel de configuration du module USR-DR154 et une infrastructure de déploiement automatisée ont été documentés et intégrés au dépôt GitHub, afin de faciliter la reprise et la continuité du projet. Les prochaines étapes consisteraient à finaliser le système d'alertes, à améliorer la robustesse matérielle (PCB dédié, SSD), et à réaliser un déploiement en conditions réelles sur le terrain.

Dépôt GitHub du projet

<https://github.com/LeBaptCode/PolySource> — Contient l'ensemble du code source (firmware ESP32, configuration Grafana, docker-compose), la documentation des séances, les tutoriels de configuration et les ressources techniques (datasheets, manuels AT).